

# Assignment 2: A Multi-Level Feedback Queue Scheduler

MTL458: Operating Systems

Semester I, 2026-2027

Deadline: [Deadline Date]

## Introduction

In this assignment you will implement a Multi-Level Feedback Queue (MLFQ) scheduler that schedules real processes. Your scheduler reads an input file, launches each job as a child process, and decides which one holds the CPU at every moment by stopping and resuming them with signals.

Your program prints a log of the scheduling decisions it made to standard output. It writes no files.

## Environment and Build Requirements

- Language: C, compiled with GCC 11.1 or later under the C17 standard. Do not install or link against any third-party library.
- Your scheduler takes the path of an input file as its single command line argument.
- Your program must terminate on its own once every job has finished, leaving no child process running and no unrealed child.
- You must schedule the jobs yourself. A program that computes the schedule without launching and controlling the child processes will receive no credit.

## The Supplied Workload Program

You are given *workload.c* and a compiled *workload* binary. This is the program your scheduler runs as each job. Do not modify it.

```
./workload <name> <cpu_ms> <io_every_ms> <io_dur_ms>
```

Its behaviour, which your scheduler must cooperate with:

1. On startup it sends itself *SIGSTOP* and parks. It does not run until your scheduler resumes it.
2. It consumes *cpu\_ms* milliseconds of CPU. It measures this as elapsed time during which it was not stopped, so a job needs the same amount of CPU regardless of how often you preempt it.
3. Every *io\_every\_ms* of consumed CPU it performs an I/O. It writes the line *IO <name>* to **file descriptor 3**, sleeps for *io\_dur\_ms*, writes the line *RDY <name>*, and then parks itself with *SIGSTOP* again. An *io\_every\_ms* of 0 means the job performs no I/O.
4. When it has consumed all of its CPU it writes the line *DONE <name>* and exits.

Your scheduler provides that descriptor to each child and reads what the jobs write to it. Every message is one line ending in a newline, and all jobs share the descriptor. A message is short enough that two jobs' writes are never interleaved within a line, but a single read may return several lines, or only part of one.

Stopping and resuming are not instantaneous. A job that has just written *RDY* is about to park itself, and a job you have just sent *SIGSTOP* is about to stop. Resuming a job that has not yet stopped leaves it stopped, and it

will never run again, so establish that a job really has stopped before you resume it.

A job performing I/O is not on the CPU, but it is not stopped either. Its I/O proceeds while another job runs. Do not send it *SIGSTOP* during that period.

## The Scheduling Policy

There are  $n$  ready queues,  $Q0$  up to  $Q(n-1)$ , where  $Q(n-1)$  is the highest priority and  $Q0$  is the lowest. Each queue is a FIFO.

Each queue has a **quantum**, measured in milliseconds, and an **allotment**, measured in quanta. The quantum is the length of one uninterrupted turn on the CPU. The allotment is the number of such turns a job may take at that level before its priority is reduced.

Each job carries two counters: the milliseconds left in its current turn, and the quanta it may still use at its current level. Both are set to the values of a queue when the job arrives at that queue. The turn counter is refilled only when it reaches zero, on demotion, and on a priority boost. It is not refilled when a job blocks for I/O, and not when a job loses the CPU to a higher priority job.

The scheduler follows these rules:

- **Rule 1:** If  $\text{Priority}(A) > \text{Priority}(B)$ , A runs and B does not.
- **Rule 2:** If  $\text{Priority}(A) = \text{Priority}(B)$ , A and B run in round robin using the quantum of their queue.
- **Rule 3:** When a job enters the system, it is placed in the highest priority queue.
- **Rule 4:** Once a job uses up its allotment at a given level, regardless of how many times it has given up the CPU, its priority is reduced and it moves down one queue.
- **Rule 5:** After every  $S$  milliseconds, move every job in the system to the topmost queue.

## Input

All values are non-negative integers, and fields on a line are separated by single spaces. All times are in milliseconds.

```
n S G
q[n-1] q[n-2] ... q[0]
a[n-1] a[n-2] ... a[0]
m
name arrival cpu io_every io_dur
... m job lines in total ...
```

- $n$  is the number of queues.  $S$  is the priority boost period, and 0 disables boosting.  $G$  is the gaming tolerance flag, described in Part 4.
- The second line gives each queue's quantum. The third gives each queue's allotment, as a number of quanta. Both lines run from the highest priority queue to the lowest, so the first value belongs to  $Q(n-1)$  and the last to  $Q0$ .
- $m$  is the number of jobs, followed by one line per job. On a job line:
  - *name* identifies the job, and is the name you print in the log.
  - *arrival* is the time at which the job enters the system.
  - *cpu* is the total CPU time the job needs before it finishes.

- *io\_every* is how much CPU the job consumes between one I/O and the next. A value of 0 means the job performs no I/O.
- *io\_dur* is how long each of that job's I/O operations takes.

```
3 0 1
500 1000 2000
2 2 2
2
A 0 2600 0 0
B 0 1700 750 300
```

## Output

Print one line to standard output for every scheduling event, each beginning with the time in milliseconds. Time starts at zero at the moment your scheduler begins scheduling, once all setup is complete, so the cost of starting the jobs is not counted against it. Arrival times and the boost period in the input are measured from the same origin. The timestamps are not compared exactly. What is graded is the sequence of decisions, so small run to run variation in the times is expected and acceptable.

| Line                           | Printed when                                                                            |
|--------------------------------|-----------------------------------------------------------------------------------------|
| <i>t ARRIVE name</i>           | The job enters the system and is placed in $Q(n-1)$ .                                   |
| <i>t RUN name Qk</i>           | You dispatch this job, which is at level $k$ .                                          |
| <i>t IDLE</i>                  | No job is runnable. Printed once when the CPU becomes idle, not repeatedly.             |
| <i>t QEXPIRE name Qk</i>       | The job used a full quantum at $Qk$ and allotment remained, so it stays at $Qk$ .       |
| <i>t DEMOTE name Qfrom Qto</i> | The job used a full quantum and its last unit of allotment, so it moves down one queue. |
| <i>t IO_START name</i>         | The job announced an I/O and left the CPU.                                              |
| <i>t IO_END name</i>           | The job's I/O completed and it rejoined a ready queue.                                  |
| <i>t BOOST</i>                 | A priority boost occurred.                                                              |
| <i>t FINISH name</i>           | The job announced that it had completed.                                                |

Print a *RUN* line every time you give the CPU to a job, including when the job you pick is the one that was already running.

## Clarifications

1. A job that finishes gives up the CPU immediately, and produces a *FINISH* line alone. Never print a *QEXPIRE* or *DEMOTE* for a job that has finished, including when it finishes exactly as its quantum runs out.
2. A job that loses the CPU to a higher priority job keeps both of its counters and returns to the **head** of its own queue. Losing the CPU this way is not itself an event, and is visible in the log only as the *RUN* line of the job that takes over.
3. A job returning from I/O joins the **tail** of the queue it was in when it blocked.
4. A job arriving at a strictly higher priority than the running job takes the CPU immediately. A job arriving at the same or lower priority does not.

5. A boost takes effect as soon as it is due. Park the running job, move every job including the running and blocked ones to  $Q(n-1)$ , refill every job's counters, and then dispatch again. Boosts occur at every positive multiple of  $S$ , measured from time zero, not from the previous boost.
6. Jobs placed into  $Q(n-1)$  by a boost are ordered by their position in the input file. A job blocked on I/O changes level immediately and joins the tail of  $Q(n-1)$  when its I/O completes.
7. If several events fall at the same instant, print them in this order: *FINISH*, *IO\_START*, *IO\_END*, *QEXPIRE*, *DEMOTE*, *ARRIVE*, *BOOST*, and last *RUN* or *IDLE*. If two events of the same kind fall at the same instant, order them by the position of the job in the input file.
8. A job at  $Q0$  is never demoted. When it exhausts its allotment there, refill the allotment, print *QEXPIRE*, and send it to the tail of  $Q0$ .
9. You may create the child processes at any time, provided that no job consumes CPU before its arrival time.
10. When the last job finishes, print nothing further and exit.

### Part 1: Process Control and a Single Queue (25 points)

Implement the mechanism. Run each job as a child process, give the CPU to exactly one of them at a time, and clean up each child when it exits.

For this part every input has  $n$  equal to 1, every *io\_every* is 0,  $S$  is 0, and every job arrives at time 0, so the policy is plain round robin.

```

Input:
1 0 1
500
2
2
A 0 1250 0 0
B 0 750 0 0

Expected decision log:
0 ARRIVE A
0 ARRIVE B
0 RUN A Q0
500 QEXPIRE A Q0
500 RUN B Q0
1000 QEXPIRE B Q0
1000 RUN A Q0
1500 QEXPIRE A Q0
1500 RUN B Q0
1750 FINISH B
1750 RUN A Q0
2000 FINISH A

```

### Part 2: Multiple Queues, Quanta and Allotments (30 points)

Add the queues. Implement Rules 1 to 4, with a separate quantum and allotment per queue.

Input:

3 0 1

300 600 1200

2 2 2

2

A 0 2200 0 0

B 0 850 0 0

Expected decision log:

0 ARRIVE A

0 ARRIVE B

0 RUN A Q2

300 QEXPIRE A Q2

300 RUN B Q2

600 QEXPIRE B Q2

600 RUN A Q2

900 DEMOTE A Q2 Q1

900 RUN B Q2

1200 DEMOTE B Q2 Q1

1200 RUN A Q1

1800 QEXPIRE A Q1

1800 RUN B Q1

2050 FINISH B

2050 RUN A Q1

2650 DEMOTE A Q1 Q0

2650 RUN A Q0

3050 FINISH A

A job is not demoted at the end of every turn. It is demoted at the end of the turn that spends its last unit of allotment. With an allotment of 2 it therefore takes two full turns at a level before dropping.

Do not hardcode the number of queues, the quanta or the allotments.

### Part 3: I/O and Staggered Arrivals (25 points)

Handle jobs that block and jobs that arrive after the run has begun.

A job that announces an I/O leaves the CPU immediately, part way through its turn. Neither of its counters is refilled, so when it returns it resumes the turn it was part way through rather than starting a fresh one. While it is blocked the CPU goes to the highest priority runnable job, or is idle if there is none.

Input:

3 0 1

500 1000 2000

2 2 2

2

A 0 2600 0 0

B 0 1700 750 300

Watch what happens to *B* in the middle of this run:

```
1500 RUN B Q2
1750 IO_START B
1750 RUN A Q1
2050 IO_END B
2050 RUN B Q2
2300 DEMOTE B Q2 Q1
```

*B* blocks 250 milliseconds into a 500 millisecond turn, with 250 still owing. It preempts *A* on returning, because it comes back to a strictly higher queue, and then runs exactly the 250 milliseconds it had left before being demoted. Performing I/O bought it nothing: its turn resumed where it left off rather than starting again, which is Rule 4 doing its work. If every job is blocked at once the CPU has nothing to run, and your scheduler must print *IDLE*.

## Part 4: Priority Boost and Gaming Tolerance (20 points)

This part has two additions, independent of each other.

### Priority boost

Implement Rule 5. When  $S$  is greater than 0, every job in the system returns to  $Q(n-1)$  at every multiple of  $S$ , so a job that has sunk to the bottom is periodically given another chance at the top. Without it, a long running job can be starved indefinitely by shorter ones.

```
1600 DEMOTE B Q2 Q1
1600 RUN A Q1
1900 BOOST
1900 RUN A Q2
```

Both jobs had worked their way down to  $Q1$ . The boost at 1900 lifts them back to  $Q2$  with full counters, and the cycle begins again.

### Gaming tolerance

Rule 4 is not the rule MLFQ originally used. The original scheme had two rules in its place:

- **Rule 4a:** If a job uses up its entire quantum while running, its priority is reduced and it moves down one queue.
- **Rule 4b:** If a job gives up the CPU before its quantum is up, it stays at the same priority level and starts again with its counters full.

Under those rules a job that always blocks just before its quantum expires is never demoted, and can hold a share of the CPU far larger than it deserves. Rule 4 removes that behaviour, and the difference is known as gaming tolerance.

Your scheduler must implement both, selected by the flag  $G$ :

- $G$  equal to 1 uses Rule 4, the behaviour required in Parts 1 to 3. A job that blocks for I/O keeps both counters.
- $G$  equal to 0 uses Rules 4a and 4b. A job that announces an I/O has both counters refilled to the values of its current queue at that moment.

Nothing else changes between the two modes. Run the following input with  $G$  set to 1 and then to 0, and compare the two logs. Under Rule 4 the job *GAMER* is demoted. Under Rules 4a and 4b it never is, and it keeps a share of the CPU it has not earned.

```
Input:
2 0 1
800 1600
2 2
2
A 0 4000 0 0
GAMER 0 2400 650 350
```

## Assumptions and Bounds

- $1 \leq n \leq 8$  and  $1 \leq m \leq 16$ .
- Every quantum is between 50 and 2000 milliseconds. Every allotment is between 1 and 8 quanta.
- $S$  is either 0, or between 200 and 10000.
- Arrival times are at most 10000 milliseconds and every *cpu* is between 50 and 10000 milliseconds. No test case runs longer than 60 seconds.
- Job names are unique, are at most 15 characters long, and contain only letters and digits.
- *io\_dur* is at least 20 whenever *io\_every* is greater than 0.
- Every job runs to completion. No job fails to start and no job exits with an error.
- The input file is always well formed. You do not need to validate it.

## Submission

- This assignment is to be done in groups of at most 3. Register your group on Gradescope before submitting. One member submits on behalf of the group and adds the other members, after which the grade appears in every member's account.
- Submit a single C source file named *a2.c*.
- Ensure that your code is well commented and follows accepted practice for readability and maintainability.
- Collaboration is permitted within a group and is not permitted between groups. All submissions will be strictly checked for plagiarism, including comparisons with submissions from previous years as well as AI-generated content. Any instance of plagiarism will result in a score of zero for the assignment.

Good luck, and happy coding!